



INSTITUTE OF SPACE TECHNOLOGY
KICSIT, Kahuta Campus

Project Proposal:

“NEXIUM - Digital Crime Control”

Using C++

Group Members

Fatima Sulman – 242201010

Musaib Ahmed – 242201022

Abeera Zainab – 242201025

Submitted to: Ms Asra Nadeem



Overview

Imagine a **top-secret high-tech prison**, where the world's most dangerous criminals are kept under constant watch. Every cell is monitored, every criminal record is updated in real-time, and no detail is left unchecked. This is **NEXIUM**—a digital command center for crime agencies.

As an authorized agent, you can log in and instantly:

- See which criminals are captured and which are still at large
- Check which prison cells are empty and which are occupied
- Track the duties of each jailer, ensuring the prison runs smoothly
- Search criminal records and uncover their digital fingerprints

Built using **linked lists, arrays, and queues**, NEXIUM turns a complex, high-security environment into a **manageable, interactive system**, giving agents the power to stay one step ahead of crime.

Problem Statement

In a world where high-profile criminals operate with advanced networks, **manual prison management is no longer enough**. Agencies struggle to:

- Track who is captured and who is still at large
- Monitor which cells are empty or occupied in real-time
- Rotate jailers efficiently without confusion
- Keep accurate records of criminal activity for investigations

Without a **centralized digital system**, errors and delays can lead to security risks, escapes, or mismanagement. **NEXIUM** solves this by transforming the prison into a **high-tech, interactive command center**, where every criminal, cell, and duty is monitored and controlled seamlessly.

Objectives

- Apply **linked lists, arrays, and queues** in a practical system
- Search and track criminal records efficiently
- Monitor prison cell occupancy dynamically
- Automate jailer duty rotation
- Implement a secure login system for authorized agents
- Develop teamwork and problem-solving skills

Scope of the Project

The system allows users to:

- Search criminals by ID
- View all criminal records (Captured / At Large)
- Display prison cell occupancy and empty cells
- Show the current jailer on duty and rotate duty automatically
- Menu-driven, interactive, beginner-friendly console system

Why C++ is Suitable for This Project

C++ is well-suited for this project because it aids

- **Memory management:** Easy handling of dynamic structures (linked lists)
- **Efficiency:** Fast execution suitable for real-time simulation
- **Data structures support:** Arrays, stacks, queues, and pointers work natively
- **Beginner-friendly console I/O:** Makes simulation interactive and easy to implement

Data Structure Usage

Feature	Data Structure	Purpose
Criminal Records	Linked List	Dynamic addition, search, and traversal of criminals
Prison Cells	Array	Fixed-size storage of cells, easy indexing and status check
Jailer Duty	Queue	Rotate jailer duty fairly in FIFO order

Methodology

- **Login System:** Only authorized agents can access NEXIUM
- **Linked List for Criminals:**
 - Store criminal ID, name, and status
 - Enable dynamic search by ID
- **Array for Prison Cells:** Track occupancy and display empty cells
- **Queue for Jailer Duty:** Rotate duty automatically, ensuring fair scheduling
- **User Interaction:**
 - Menu-driven console interface
 - Options: View criminals, search criminals, view cells, check jailer duty, exit

Expected Output

- Search for criminals by ID with instant results
- View all criminal records with their current status
- Display prison cell occupancy clearly
- Show which jailer is on duty
- Interactive console interface simulating a high-tech prison

Implementation & Technical Details

- **Linked List:** Stores criminal data dynamically; supports search operation
- **Array:** Maintains fixed number of prison cells; indexes show occupancy
- **Queue:** Rotates jailer duty using FIFO logic
- **Login System:** Basic username/password verification
- **Menu System:** Console-based options for agent interaction
- **Dynamic Memory:** Linked list nodes allocated and deallocated to prevent memory leaks

Example Testing & Validation Table

Action	Linked List State	Array State (Cells)	Queue State (Jailer)	Result
Search Criminal ID 1	Rex	Cell 1: Rex	Ali	Correct
Search Criminal ID 2	Nova	Cell 2: Empty	Ali	Correct
View Prison Cells	Rex, Nova	Cell 1: Rex, Cell 2: Empty, Cell 3: Empty	Ali	Correct
Rotate Jailer Duty	-	-	Sara (after rotation)	Correct

Edge Cases Handled:

- Searching for a non-existent criminal ID
- Empty prison cells
- Queue rotation when duty completes

Team Contributions:

Team Member	Core Responsibility	Assigned Modules
Fatima Sulman	Linked list implementation & testing	- Criminal Records (Linked List) - Search Criminal by ID - Display Criminal Details
Abeera Zainab	Queue implementation & navigation logic	- Jailer Duty Rotation (Queue) - Menu Navigation - Option Handling (View Jailers, Rotate Duty)
Musaib Ahmed	Array implementation & system security	- Prison Cells (Array) - Display Cell Occupancy - Login System (Username/Password) - Output Formatting / User Interaction

Expected Future Enhancements

- ***File Handling & Database Integration***
Store criminal records, cell status, and jailer duty in files or a database for permanent storage and retrieval.
- ***Threat Level Classification***
Assign High / Medium / Low threat levels to criminals to prioritize monitoring and management.
- ***Graphical User Interface (GUI)***
Build a visual dashboard showing prison cells, criminals, and jailer schedules for better usability.
- ***Real-Time Alerts & Notifications***
Alert agents automatically if a criminal escape.